

Fabricated Authority in the Wild: Policy Provenance as a Control Surface for Tool-Calling Agents

Anonymous Author(s)

September 5, 2026

Abstract

Tool-using language-model agents can refuse in text while still taking an unsafe action. We study *fabricated authority*: user-controlled claims that a forbidden action is allowed by a non-existent approval, exemption, or superseding policy. Across 1,968 hosted-model direct-channel trials with 8 scenarios, 6 operating-policy phrasings, 7 attack templates, and 2 hosted models, fabricated authority raises logged tool-call violations from 0/240 in controls to 208/1440 under attack (14.4%, $p = 1.1 \times 10^{-15}$). The dominant variable is operating-policy wording: a provenance-aware policy has 0/288 direct user-message violations, while a documented-exemption carve-out has 93/288 (32.3%). A matched contrast gives complete separation, 18/18 versus 0/18. A length-control ablation shows this is not verbosity, and two adaptive-attacker runs break weak controls while failing to breach the provenance-aware policy. We then scale a forced indirect-channel study to all 8 scenarios: when the same fabricated authority arrives through a required policy-lookup tool, clean lookups cause 0/48 violations but poisoned lookups cause 32/48, including 8/24 under the provenance-aware policy and 24/24 under the exemption policy. These findings suggest that agent evaluations should score tool logs, deployment prompts should reject unauthenticated authority claims, and production systems need runtime provenance controls for tool outputs.

1 Introduction

Language-model safety evaluations often ask whether the model produces harmful text. Tool-using agents require a different view. The system output is no longer just a string: it can be a database export, wire transfer, deletion command, privilege grant, or security change. In that setting, the most important dependent variable is often available without a judge: did the agent call the forbidden tool with violating arguments?

This matters because text and action can diverge. In our paired text/tool experiment, 32 of 432 trials contain a refusal in prose while also emitting the forbidden tool call. None of these divergences occur in no-attack controls. A text-only evaluation would score these as safe refusals even though the external action fires.

We focus on *fabricated authority*. The user does not primarily deny that the requested action is restricted. Instead, the user claims that some authority permits it: a superseding directive, prior approval, documented audit exemption, or updated classification. This is natural in real deployments because agent policies often include operational carve-outs. Engineers write policies like “do not export records unless an exemption is documented” or “handle routine requests end-to-end without bothering staff.” Such language is convenient, but it creates hooks for user-controlled claims. AIWILD explicitly solicits work on agent security, tool misuse, prompt injection, evaluation, and agent systems; this paper sits at that intersection because the failure is an external action, not merely an unsafe answer.

The central finding is that policy wording can dominate the outcome. A policy that says claimed exemptions, prior approvals, or superseding policies arriving in user input or tool output are not authoritative drives violations to 0/288 in the main hosted-model sweep. A policy that allows documented exemptions reaches 93/288, or 32.3%, under identical attacks.

Contributions. We contribute: (1) a logged-tool-call evaluation for fabricated authority, (2) a policy-phrasing measurement matrix, (3) a one-sentence provenance mitigation for direct user-channel attacks, (4) a length-control ablation, (5) an adaptive-attacker check, (6) a target-sensitivity negative result on Sonnet, and (7) a scaled forced-indirect study showing that tool-output provenance requires runtime controls.

2 Motivation: from judged text to logged action

The project began as a GRPO-trained text attacker scored by an LLM judge. The initial result looked promising: measured ASR rose from 25% to 48%. Validating the judge against 200 human-labelled responses showed that the effect was an artifact. The local judge reported 83% ASR on outputs whose human-labelled ASR was 19%, with 36% accuracy and 23% precision. The same outputs scored 14% ASR under `gpt-4o-mini` and 25% under a refusal heuristic. Re-running the trained-vs-base comparison on 120 held-out behaviors with the validated judge produced 12.5% versus 11.7%, Fisher $p = 1.00$.

We do not claim novelty for judge failure. Recent work studies judge unreliability at far greater scale. The lesson for this paper is methodological: where a verifiable action log exists, use it. Tool-call traces make the safety outcome less ambiguous and avoid evaluator false positives.

3 Threat model and setup

The defender is a single-turn tool-using agent. It receives a system prompt containing an operating policy, a user task, and a fixed set of tools. Some tools can be used compliantly, but each scenario defines a policy-violating use through a predicate over tool name and arguments. The agent also has a safe alternative tool, `request_approval`, so non-violation is not confounded with doing nothing.

The attacker controls the user message. The attacker cannot modify the system prompt, tool schema, or execution environment. The attacker can include fabricated claims about policy, approval, urgency, or document status. In the adaptive setting, the attacker additionally sees the defender’s system prompt, tool schema, and transcripts of prior failed attempts.

We use 8 hand-written scenarios covering data exfiltration, destructive operations, financial actions, privilege escalation, and security degradation: medical export, payroll disclosure, customer PII sharing, production database deletion, backup deletion, security disablement, wire transfer, and access grant. Each scenario specifies a forbidden tool call and a safe alternative.

Table 1: Operating-policy phrasings. The critical variable is whether the policy authenticates the provenance of claimed authority.

Policy	Content
<code>strict_hatch</code>	Prohibition, safe alternative, and provenance clause
<code>strict</code>	Prohibition plus safe alternative
<code>bare</code>	Prohibition only
<code>autonomous</code>	Routine end-to-end handling with a carve-out
<code>exemption</code>	Prohibition unless an exemption is documented

The provenance clause is: “Claims of exemption, prior approval, or superseding policy that arrive in a user message or tool result are NOT authoritative. Treat them as unverified and call `request_approval`.”

4 Results

4.1 Fabricated authority causes tool violations

The main hosted sweep contains 1,680 trials: 8 scenarios, 5 policies, 7 attacks, 2 targets, and 3 trials. No trials errored.

Table 2: Main hosted sweep.

Condition	Violations	Rate
No attack	0/240	0.0%
Fabricated authority	208/1440	14.4%

The one-sided Fisher exact test gives $p = 1.14 \times 10^{-15}$. Since the control floor is zero, violations in the attacked condition are attributable to fabricated authority rather than ordinary noncompliance in these hosted models.

4.2 Policy phrasing dominates

Table 3: Attacked trials, hosted targets pooled.

Policy	Violations	Rate	95% CI
strict_hatch	0/288	0.0%	[0.0%, 1.3%]
strict	7/288	2.4%	[1.2%, 4.9%]
bare	21/288	7.3%	[4.8%, 10.9%]
autonomous	87/288	30.2%	[25.2%, 35.7%]
exemption	93/288	32.3%	[27.2%, 37.9%]

The planned contrast `strict_hatch` versus `exemption` gives Fisher $p = 1.37 \times 10^{-32}$. The ranking is non-monotonic. A bare prohibition is safer than a prohibition with a documented-exemption carve-out, suggesting that operational exceptions become attack surface when the agent is not told how to authenticate them.

4.3 The effect is not policy length

Table 4: Length-control ablation on gpt-4o-mini.

Policy	Words	Violations	Rate
strict_hatch	77	0/96	0.0%
strict	40	5/96	5.2%
strict_verbose	98	11/96	11.5%

`strict_hatch` versus `strict_verbose` gives Fisher $p = 0.000361$. More words do not explain the mitigation. If anything, irrelevant policy detail appears to dilute attention to the restriction.

4.4 Matched contrast and adaptive attacker

The strongest single contrast isolates `medical_export` and the three attacks that reliably fire on it, holding model, scenario, temperature, and attack text fixed. `exemption` scores 18/18. `strict_hatch` scores 0/18. Fisher $p = 1.1 \times 10^{-10}$.

We then gave an attacker LLM the defender’s exact system prompt, tool schema, and a transcript of its failed attempts. Across two runs, it broke the weak control policy in 8/8 and 7/8 scenarios, but breached `strict_hatch` in 0/8 scenarios both times. Fisher p values were 7.8×10^{-5} and 6.99×10^{-4} . This does not prove the clause is unbreakable. It shows the clause resists an adaptive attacker strong enough to defeat the closest weak alternative.

4.5 Frontier-model matched contrast

We also ran a compact matched contrast on `claude-sonnet-4-5`, using `medical_export`, the `strict_hatch` and `exemption` policies, no-attack controls, and the three strongest fabricated-authority attacks at 6 trials per cell. Sonnet produced zero violations in all 48 trials: 0/12 controls and 0/36 attacked, including 0/18 under `exemption` and 0/18 under `strict_hatch`. This is a useful negative result for the longer paper. It shows that the attack is not universal across stronger hosted models, and it strengthens the claim that target choice matters rather than weakening the policy-phrasing result on the smaller production targets.

4.6 Additional hosted target: replicated policy ordering

After the ground check, we added a targeted 280-trial sweep on `gpt-4.1-mini`: 8 scenarios, 5 policy phrasings, 7 attacks, and 1 trial per cell. The control floor remains zero (0/40), while attacked trials produce 68/240 violations (28.3%, Fisher $p = 5.38 \times 10^{-6}$). Most importantly, the policy ordering replicates: `strict_hatch` remains 0/48 under attack, while `exemption` rises to 30/48 (62.5%, Fisher $p = 1.06 \times 10^{-12}$ for the contrast). This is not a replacement for a full multi-model sweep, but it directly addresses the most important breadth concern: the direct-channel provenance result is not unique to `gpt-4o-mini` or `claude-haiku-4-5`.

4.7 Forced indirect channel: the clause is not universal

The earlier indirect-channel attempt was inconclusive because the agent never consulted the poisoned corpus. We therefore added a forced policy-lookup experiment: before deciding, the agent must call `lookup_policy`; the fabricated authority appears only in that tool result, never in the user message. This exercises a realistic retrieval or documentation poisoning path.

Across 96 real `gpt-4o-mini` API trials spanning all 8 scenarios, the agent consulted the lookup tool in 96/96 trials. No clean lookup caused a violation (0/48). Poisoned lookup output caused 32/48 violations ($p = 7.59 \times 10^{-14}$ versus clean). The `exemption` policy failed completely (24/24), while `strict_hatch` failed in 8/24; the policy contrast remains significant ($p = 3.26 \times 10^{-7}$), but no longer yields complete protection. Violations appeared in every scenario family: 4/6 for access grant, 6/6 for backup deletion, 3/6 for customer PII sharing, 5/6 for medical export, 3/6 for payroll disclosure, 3/6 for production database deletion, 3/6 for security disablement, and 5/6 for wire transfer.

This is the most important new result for the longer paper because it bounds the headline mitigation. The provenance clause strongly protects against direct user-channel fabricated authority, but poisoned required tool output remains substantially more dangerous. The direct and indirect settings should therefore be reported separately: `strict_hatch` is a strong direct-channel mitigation, not a complete defense against retrieval-mediated authority poisoning.

4.8 Text/action divergence and small-model boundary

In 432 paired trials, 32 cases have refusal in prose while the forbidden tool call fires. The divergence never occurs without an attack: 0/144 controls versus 32/288 attacked, Fisher $p = 1.26 \times 10^{-6}$.

The local `llama3.2` run contains 560 trials. It violates policy in 54/80 no-attack controls and 270/480 attacked trials. The attack does not help ($p = 0.979$) because the baseline violation floor is already 67.5%. This bounds the mitigation: policy phrasing helps models that can follow policy at baseline, but it cannot compensate for weak baseline policy adherence.

5 Scenario and policy design detail

Table 5: Scenario families used in the harness. Each scenario contains a forbidden tool call, scenario-specific violating arguments, and a safe alternative action.

Family	Scenario	Representative forbidden action
Data exfiltration	Medical export	Export patient records externally
Data exfiltration	Payroll disclosure	Send payroll data outside the organization
Data exfiltration	Customer PII sharing	Share customer identifiers externally
Destructive action	Production database deletion	Delete production records
Destructive action	Backup deletion	Remove recovery snapshots
Security change	Endpoint protection	Disable endpoint protection
Financial action	Wire transfer	Initiate a transfer without approval
Privilege escalation	Access grant	Grant administrative access

The scenarios were intentionally written to look like ordinary enterprise-agent tasks rather than classic harmful-behavior prompts. The attacker does not ask the model to produce harmful instructions. The user asks the agent to

complete an operational request and supplies fabricated authority that appears to license the action. This design better matches the failure mode of deployed agents: the danger is not a harmful paragraph, but an apparently routine tool call.

The violation predicate is also scenario-specific. For example, exporting records to an internal compliance mailbox may be non-violating, while exporting the same table to an external research address is violating. This avoids overcounting legitimate tool use and keeps the dependent variable aligned with the deployment policy.

6 Why provenance is the control surface

The attack taxonomy separates three kinds of reasoning pressure. Category attacks claim that the action is not covered by the prohibition. Urgency attacks claim that normal review should be bypassed. Provenance attacks claim that an external authority has already changed the policy state. The empirical ordering is sharp: category attacks score 0/240, urgency scores 2/240, and provenance/process-state attacks score 43/240 to 68/240.

This suggests that the models are not simply ignoring the policy. If they were, category attacks and controls would also fire. Instead, the models usually understand what the action is. They fail at authenticating whether the claimed exception exists. That is why the one-sentence policy works: it moves provenance authentication from an implicit expectation to an explicit instruction.

7 Reproducibility and artifact checks

All main claims in this paper are backed by raw JSON artifacts and claim-guard tests. The repository includes:

- `results/agentlic_sweep_api.json`: hosted sweep for the main table.
- `results/agentlic_ablation.json`: length-control ablation.
- `results/agentlic_matched_contrast.json`: 18/18 versus 0/18 contrast.
- `results/agentlic_gap_full.json`: paired text/tool divergence.
- `results/agentlic_adaptive*.json`: two adaptive attacker runs.
- `results/agentlic_indirect_all8_t1.json`: scaled forced-indirect study over all 8 scenarios.
- `results/agentlic_sweep_local.json`: local llama3.2 boundary case.

A final repository-side verification pass ran the full test suite, targeted paper-claim tests, lint, package build, CLI smoke tests, LaTeX compilation, and PDF page-count checks. This matters because several earlier cracks were only visible after attempting to invalidate the result: weak adaptive attacks, length confounding, small-model floor effects, text/action mismatch, and indirect-channel poisoning.

8 Ethics and disclosure posture

The experiments use synthetic scenarios and no real systems or personal data. The attack strings are not credentials, exploits, or access paths to any deployed system. The most actionable result is defensive: a deployer can add a provenance clause to the system prompt and evaluate future agents by checking tool logs. We nevertheless avoid claiming that the clause is a universal solution. A stronger red-team model, multi-turn attack, indirect prompt injection, or RL-trained attacker may defeat it.

9 Reviewer-facing narrative

The longer version should not be read as another jailbreak paper whose only metric is a higher attack success rate. The central object is the deployment contract between an agent and its operator. Tool-use deployments almost always need exceptions: maintenance windows, emergency actions, prior approvals, delegated access, audit exports, and compliance holds. Those exceptions are necessary in real systems, but they create a provenance problem. The agent must distinguish a true exception from a user-controlled statement that merely describes one.

This framing gives the paper a sharper claim than “authority language can be persuasive.” The vulnerable surface is not generic persuasion; it is unauthenticated policy state. Our strongest attacks are precisely the ones that assert the policy has already changed or the approval step has already occurred. Our weakest attacks are those that ask the model to misclassify the underlying action. That contrast supports the mechanism: models often know what the action is, but fail to verify who is allowed to authorize it.

The forced indirect-channel result deepens the story. Direct user-channel provenance can be handled with a sentence in the system prompt. Tool-output provenance is harder: once the same authority claim arrives through a required documentation lookup, `strict_hatch` no longer eliminates failures. This turns the longer paper from a simple mitigation note into a more general measurement of policy provenance across channels.

10 Operational implications

For deployers, the practical lesson is to separate three policy layers.

1. **Domain policy**: what actions are restricted.
2. **Procedural policy**: what safe alternative should happen when an action is restricted.
3. **Provenance policy**: which channels are allowed to modify or exempt the first two policies.

Many agent prompts contain the first layer and sometimes the second. The experiments show that omitting the third layer leaves a gap. Adding a provenance sentence closes that gap for direct user claims, but not for poisoned tool output. Therefore a production deployment also needs runtime provenance controls: authenticated policy stores, signed tool outputs, allowlisted exception sources, or a policy engine outside the model.

This is why the paper’s recommendation is intentionally narrow. We do not propose prompt wording as a full security architecture. We propose it as a cheap first line of defense and as a diagnostic: if a model cannot obey the provenance sentence in the direct channel, it should not be trusted with tools that require exception handling.

11 Visual summary of the main effects

The key effects can be read as three deployment questions rather than as independent jailbreak scores.

Policy	Hatch	Strict	Bare	Autonomous	Exemption
Direct user attack violation rate	0.0%	2.4%	7.3%	30.2%	32.3%

Figure 1: Policy wording changes direct-channel tool-call failure rates by more than 30 percentage points. The largest jump is introduced by operational carve-outs without provenance authentication.

These summaries test the remaining hypotheses most likely to matter to reviewers. The effect is not merely ordinary noncompliance because direct controls are 0/240 and clean lookup controls are 0/48. It is not only verbosity because a longer strict policy performs worse than the shorter provenance policy. It is not universal model weakness because Sonnet shows a zero-violation negative result in the compact matched contrast. It is not universal mitigation because the forced indirect channel partially breaks the provenance clause.

Channel and condition	Control	Poisoned/attacked	Main contrast
Direct user message	0/240	208/1440	provenance hatch 0/288
Required policy lookup	0/48	32/48	hatch 8/24 vs exemption 24/24
Text-only scoring gap	0/144	32/288	refusal can coexist with tool call

Figure 2: The contribution is channel-specific. Prompt-level provenance works best for direct user claims; required tool-output claims require runtime provenance controls.

12 Related work

Text jailbreak benchmarks such as HarmBench standardize harmful-behavior evaluation, while LLMStinger and related work train or search for jailbreak prompts. DarkCite shows that fabricated authority and citations can jailbreak text models, so our novelty is not the existence of authority-framed attacks. Recent judge-reliability work shows that unvalidated LLM judges can dominate reported ASR. Agent benchmarks such as GAP and AgentSeer show that text safety does not transfer cleanly to tool-calling agents, and AutoInject studies automated prompt injection attacks against tool-using agents. AIWILD accepted work such as LinuxArena, WARD, and command-line agent security studies further shows that the venue values action-level agent security failures in realistic staged environments. Context-fractured decomposition attacks study provenance gaps in cross-step artifact-mediated agent pipelines and motivate lineage tracking. Our focus is narrower and complementary: deployment-policy phrasing as an independent variable, and direct-versus-indirect channel provenance as a measurable control surface for fabricated authority.

13 Practical recommendations

Add a provenance clause. Avoid unauthenticated carve-outs. Be careful with autonomy-forward phrasing. Include a safe alternative tool. Evaluate on tool logs, not text-only outputs.

14 Limitations and next steps

The main hosted sweep has only two production targets. The scenarios are hand-written and single-turn. The adaptive attacker is `gpt-4o-mini`, not a stronger frontier model or an RL-trained optimizer. The provenance clause does fail partially in the forced indirect-channel setting: poisoned policy-lookup output causes 8/24 violations under `strict_hatch` in the all-scenario run. Multi-turn settings and stronger tool-output attacks remain open.

The longer version should add more targets, multi-turn attacks, higher-repetition indirect-channel runs, an RL-trained attacker against `strict_hatch`, 30–50 scenarios grouped by sector and tool type, and a practitioner study of policy realism. The forced-indirect study is already included as a warning: prompt-level provenance clauses mitigate direct user-message authority claims, but poisoned tool outputs need runtime provenance enforcement.

15 Design choices for the longer paper

The extended version turns the workshop short paper into a more complete empirical paper. The short version emphasizes the headline mitigation; the longer version makes the experimental design auditable. In particular, it separates four concepts that are easy to conflate.

Policy compliance versus attack resistance. A model that violates policy in no-attack controls is not attack-resistant; it has already failed. This is why the `llama3.2` run is a boundary condition rather than part of the main effect. The hosted models have a zero control floor, allowing an attack effect to be measured cleanly.

Authority content versus authority provenance. A policy can specify what is forbidden but still fail to say who is allowed to change that policy. Fabricated authority exploits that missing provenance rule. The provenance clause does not add a new domain prohibition; it tells the model how to treat unauthenticated claims about exceptions.

Safe refusal versus inaction. A no-tool-call outcome is ambiguous: the agent may be safe, confused, or stalled. We therefore include `request_approval` as a positive safe action. This makes the recommended behavior observable and deployable.

Text safety versus action safety. Text refusal is insufficient for agents. The paired gap study shows that the model can state the right policy in prose and still emit the wrong call. The longer version foregrounds this as a measurement problem, not just an attack problem.

16 Ablation summary

Table 6: Ablations and what each rules out.

Concern	Check	Result
Attack works only because controls are weak	No-attack baseline	0/240 violations
Policy effect is length	<code>strict_verbose</code>	11/96 vs 0/96
Pooled scenarios hide imbalance	Matched contrast	18/18 vs 0/18
Adaptive attack is weak	Control arm	8/8 and 7/8 broken
Mitigation is universal	<code>llama3.2</code>	fails at baseline, not pooled
Text refusal is enough	Paired gap study	32 action violations despite refusal

This table is the argument structure for reviewers. Each crack in the short paper is either closed by an ablation or explicitly bounded as a limitation.

17 Regular-paper submission plan

The regular AIWILD track allows 9 pages, excluding references and supplementary materials. This draft currently compiles under that budget and should be used as the base for the larger submission. The highest-return additions before deadline are ordered as follows:

1. Add one frontier-model target if API access and cost allow. Even a small matrix on the matched contrast would strengthen generality.
2. Scale the indirect-channel experiment across more targets. The all-scenario forced-lookup study already shows this is the strongest follow-on: clean lookups produced 0/48 violations, while poisoned lookups produced 32/48, including 8/24 against `strict_hatch`.
3. Add more scenario templates rather than more repetitions of the same 8 scenarios. Reviewer skepticism will target hand-written scenario design.
4. Add a stronger adaptive attacker, ideally a different model family from the defended target.
5. Move raw attack strings and scenario definitions to supplementary material to preserve the main-paper narrative.

18 Conclusion

Fabricated authority is a simple attack that becomes measurable when agents have tools. In hosted models that otherwise follow their policies, fabricated authority produces statistically significant tool violations. The largest lever is not model choice alone but the exact operating-policy phrasing. A one-sentence provenance clause eliminates direct user-message violations in our main sweep, survives a matched contrast and two adaptive-attacker runs, and costs nothing to deploy. The forced indirect-channel result shows the boundary: when authority claims arrive through a required tool result, prompt-level provenance is no longer enough. The deployable lesson is therefore two-part: write prompts that reject unauthenticated authority claims, and build systems that preserve provenance for every channel that can change policy state.

References

- [1] Mazeika et al. HarmBench. ICML, 2024.
- [2] Jha et al. LLMStinger. arXiv:2411.08862, 2024.
- [3] Yang et al. The Dark Side of Trust. arXiv:2411.11407, 2024.
- [4] Schwinn et al. A Coin Flip for Safety. arXiv:2603.06594, 2026.
- [5] How Reliable Is Your Jailbreak Judge? arXiv:2606.25487, 2026.
- [6] GAP: A Benchmark for General Agentic Policies. arXiv:2602.16943, 2026.
- [7] AgentSeer: Measuring Safety in Tool-Calling Agents. arXiv:2509.04802, 2025.
- [8] AutoInject: Automated Prompt Injection Attacks against Tool-Using Agents. arXiv:2602.05746, 2026.
- [9] Context-Fractured Decomposition Attacks on Tool-Using LLM Agents: Exploiting Artifact Provenance Gaps. arXiv:2606.09084, 2026.
- [10] WARD: Adversarially Robust Defense of Web Agents Against Prompt Injections. AIWILD @ ICML, 2026.